CLAUDE.md — nosqlbench

Cross-database benchmark: one social-feed workload run against Redis, MongoDB, Cassandra and Neo4j by a Rust load generator that must not be the bottleneck.

Platform: Windows 11. The engine runs natively on Windows. The databases run in Docker Desktop (Linux containers). All scripts are PowerShell.

Deadline is 3–4 days. Build milestone by milestone (see bottom). Do not start a milestone until the previous one passes its acceptance check. Prefer boring, working code over clever code.

---

0. Non-negotiable rules

1. **Never change measurement semantics without asking.** Where T0/T1 are taken, what counts as latency, warm-up handling and histogram settings are fixed by this file.
2. **Never claim a performance property you haven't measured.** If you say "overhead is X", show the command and output.
3. **No allocation in engine code on the request path.** Payloads are pre-generated. Request loops are long-lived tasks created once. Driver internals may allocate; that cost is part of client cost and is visible in the no-op calibration.
4. **No locks, channels or syscalls on the request path** except the driver call itself. Telemetry goes through a per-worker SPSC ring.
5. **Never block when the ring is full.** Increment a dropped counter and move on. Any non-zero dropped count invalidates the run.
6. **Only one database container runs at a time.**
7. **Windows only.** No Linux-only APIs, no bash, no `/sys`, no `cfg(unix)` code paths that the Windows build depends on. Use `PathBuf`, never hard-coded `/`.
8. After every change: `cargo fmt`, `cargo clippy -- -D warnings`, `cargo test`. Commit after each working step with a clear message.

---

1. Tech stack

- Rust stable (MSVC toolchain), edition 2021, single binary crate `bench`
- `tokio` (current_thread runtime **per worker**; features: rt, time, net, macros)
- `core_affinity` — pin threads
- `windows` — P/E-core detection (`GetLogicalProcessorInformationEx`) and `timeBeginPeriod(1)`
- `quanta` — high-resolution clock
- `rtrb` — SPSC ring buffers
- `hdrhistogram` — percentiles

- `rand`, `rand_chacha`, `rand_distr` — seeded RNG, Zipf
- `serde`, `toml`, `serde_json`, `csv`, `clap` (derive), `anyhow`, `thiserror`, `futures`
- Drivers: `redis` (tokio-comp), `mongodb`, `scylla`, `neo4rs`
- Databases: Docker Desktop + `docker compose`
- Scripts: PowerShell 7 (`pwsh`), falling back to Windows PowerShell 5.1 syntax
- Plots: Python (`polars`, `matplotlib`)

---

2. Layout

```
nosqlbench/
  Cargo.toml
  CLAUDE.md
  config/default.toml
  docker-compose.yml
  src/
    main.rs            # clap: `bench load`, `bench run`, `bench calibrate`
    config.rs
    cpu.rs             # P-core detection (Windows API), pinning plan, timer
resolution
    dataset.rs         # deterministic dataset generation
    workload.rs        # Op type, pre-generated op streams, body arena
    worker.rs          # per-core runtime, request loops, closed/open
scheduling
    telemetry.rs       # record packing, rings, aggregator thread
    output.rs          # summary.csv, timeseries.csv, meta.json
    adapters/
      mod.rs           # Adapter trait + factory
      noop.rs
      redis_ping.rs    # transport baseline
      redis.rs mongo.rs cassandra.rs neo4j.rs
    loaders/
      redis.rs mongo.rs cassandra.rs neo4j.rs
  scripts/
    sysinfo.ps1        # CPU, OS build, power source, power plan, Docker
version
    run_db.ps1         # up → wait healthy → load → sweep → down -v
    run_all.ps1
  plots/plot.py
  results/
```

---

3. Config (`config/default.toml`)

toml

```toml
seed = 42

[dataset]
users = 10_000
posts = 100_000
avg_follows = 20
hashtags = 1_000
max_tags_per_post = 3
zipf_s = 0.99
fanout_cap = 10_000

[workload]            # weights, normalised at load time
timeline_read = 40
user_profile_read = 25
post_like = 15
post_create = 10
hashtag_search = 5
user_follow = 5

[run]
mode = "closed"       # "closed" | "open"
concurrency = 32      # closed: total in-flight across all workers
rate = 0              # open: total target ops/s
max_inflight = 1024   # open: in-flight cap across all workers
warmup_secs = 15
measure_secs = 60
request_timeout_ms = 5000
workers = 4           # capped at available physical P-cores
ring_slots = 1_048_576

[cpu]
worker_cpus = []      # empty = auto: one logical CPU per physical P-core
aggregator_cpu = -1   # -1 = auto: first E-core, else last P-core

[endpoints]
redis = "redis://127.0.0.1:6379"
mongo = "mongodb://127.0.0.1:27017"
cassandra = "127.0.0.1:9042"
neo4j = "127.0.0.1:7687"
neo4j_user = "neo4j"
neo4j_pass = "benchpass"
```

Every field can be overridden from the CLI.

4. Dataset (deterministic)

All generation uses `ChaCha8Rng::seed_from_u64(seed)` so every loader and the op generator see the same world.

- **IDs are `u64` everywhere** (users `0..users`, posts `0..posts`). Stored as `bigint` / integer, never strings.
- **Follows:** each user follows `avg_follows` distinct targets (±50% uniform), targets drawn Zipf over users. No self-follows.
- **Posts:** author drawn Zipf over users; `created_at` spread uniformly over the 30 days before `2026-01-01T00:00:00Z`; 0..=max_tags_per_post hashtags drawn Zipf.
- **Bodies:** 40–280 ASCII chars from a fixed word list, stored in one `Vec<u8>` arena with an offsets `Vec<u32>`; adapters receive `&str` slices.
- **Hashtags:** `tag0000 .. tag0999`.
- No likes are pre-loaded.
- **Timelines** (Redis, Cassandra only): each post goes to the timeline of each follower of its author, capped at `fanout_cap`.

---

5. Workload

rust

```rust
#[derive(Clone, Copy)]
pub enum OpKind { TimelineRead, UserProfileRead, PostLike, PostCreate, HashtagS

#[derive(Clone, Copy)]
pub struct Op {
    pub kind: OpKind,
    pub user: u64,      // acting user, Zipf
    pub target: u64,    // followee (UserFollow) or post id (PostLike)
    pub body: u32,      // index into body arena (PostCreate)
    pub tags: [u16; 3],
    pub ntags: u8,
    pub new_post_id: u64,
    pub ts_ms: i64,
}
```

- Each worker gets its own pre-generated `Vec<Op>` of 1,048,576 ops (seed = `seed ^ worker_id`), cycled with a wrapping cursor.
- `new_post_id = (1 << 40) | (worker_id << 32) | seq`. Writes must be idempotent so the second pass through the stream doesn't error.
- New post timestamps: epoch + 1 day + seq ms.

- TimelineRead and HashtagSearch return **20** posts, newest first.

Op	Must do
POST_CREATE	Store post; index under each hashtag; make it visible to followers' timelines (fan-out on write for Redis/Cassandra, nothing extra for Mongo/Neo4j)
USER_FOLLOW	Record `user → target`. Idempotent. No timeline backfill
TIMELINE_READ	20 newest posts by accounts `user` follows, with body and author
POST_LIKE	Record `(user, post)` like; increment post's like count
USER_PROFILE_READ	Fetch full profile by id
HASHTAG_SEARCH	20 newest posts for tag, with body

Redis and Cassandra fan out on write; MongoDB and Neo4j fan out on read. This is deliberate: each database is benchmarked with its idiomatic design.

---

6. Adapter trait

```rust
pub trait Adapter {
    async fn execute(&self, op: &Op, bodies: &BodyArena) -> Result<(), AdapterE
}
```

- Each worker constructs **its own** adapter instance inside its own runtime.
- Static dispatch: worker is generic over `A: Adapter`; `main` matches on db name once.
- Prepare statements (Cassandra) and parameterise queries (Neo4j) in the constructor.
- Adapters must fully consume responses.
- Errors map to status codes; never panic on a DB error.

`noop.rs`: returns `Ok(())` after `std::hint::black_box(op)`. Measures engine overhead and ceiling throughput.

`redis_ping.rs`: sends `PING` for every op regardless of kind. Measures the round trip Windows → Docker VM → container → back, i.e. the transport floor every database pays. Reported as its own result, never mixed into Redis numbers.

---

7. Worker and scheduling

CPU and timers (`cpu.rs`)

- Call `timeBeginPeriod(1)` at startup and `timeEndPeriod(1)` on exit. Windows' default timer resolution is ~15.6 ms, which would wreck open-loop pacing.
- Detect cores with `GetLogicalProcessorInformationEx(RelationProcessorCore)`. Each core entry has `EfficiencyClass` and a group mask. The highest `EfficiencyClass` = P-cores. Pick the lowest logical CPU of each P-core (one thread per physical core).
- If every core has the same class, treat all as P.
- Assume a single processor group (true on these laptops); error clearly otherwise.
- Print the pinning plan at startup and write it to `meta.json`.
- One OS thread per worker, pinned with `core_affinity`, running a `current_thread` Tokio runtime. One aggregator thread, pinned.

Request loops

Inside a worker, create C long-lived async loops once (`join_all` over a pre-built Vec, or `LocalSet` + `spawn_local`). Shared per-worker state is `Rc<Cell<..>>` / `Rc<RefCell<Producer>>`. No `tokio::spawn` per request.

- **Closed mode:** C = `concurrency / workers` (workers = `min(workers, concurrency)`).

```
loop {
    op   = next op from cursor
    t0   = clock.now()              // immediately before execute
    res  = adapter.execute(op).await
    t1   = clock.now()
    push record(latency = t1 - t0)
}
```

- **Open mode (coordinated-omission safe):** per-worker rate R = `rate / workers`, interval I = 1e9 / R ns, C = `max_inflight / workers`. Shared ticket counter `k`.

```
loop {
    k += 1; intended = start + k * I
    wait until intended:
        if intended - now > 2ms: tokio::time::sleep_until(intended - 2ms)
        while now < intended: tokio::task::yield_now().await
    t0  = clock.now()
    res = adapter.execute(op).await
    t1  = clock.now()
    push record(latency = t1 - intended)   // response time
    push send_lag record (t0 - intended)   // engine health
}
```

Late tickets are charged to latency. This is intended. Never re-base `start`.

Phases

Global `AtomicU8`: WARMUP → MEASURE → STOP, read with `Relaxed` when building a record. STOP: loops exit after their in-flight request.

---

8. Telemetry

Record = one `u64`:

```
bits 63..16  latency_ns (48 bits)
bits 15..8   op kind
bits  7..4   status: 0 ok, 1 error, 2 timeout
bit   3      warmup
bit   2      is_send_lag record
bits  1..0   reserved
```

- One `rtrb` ring per worker. Aggregator drains all rings, sleeping 1 ms when empty.
- Histograms: one per op kind (ok only) + one send-lag histogram; `Histogram::<u64>::new_with_bounds(1, 60_000_000_000, 3)`. Errors/timeouts counted.
- Warm-up records discarded.
- Per-second timeseries of ok/errors per op kind, bucketed by drain time.
- Throughput = measured ok ops / measure_secs.

---

9. Output

`results\<db>\<mode>_<level>_run<n>\`

- `summary.csv` `db,mode,level,run,op,count,errors,timeouts,throughput_ops_s,p50_us,p90_us,p95_us,p99_us,p999_us,max_us,send_lag_p99_us,dropped_records` plus one `op=ALL` row.

- `timeseries.csv` — `second,op,ok,errors`
- `meta.json` — resolved config, git commit, pinning plan, CPU model, Windows build, power source (AC/battery), active power scheme, Docker version and image tags, start time.
- `<op>.hgrm` — HdrHistogram percentile export.

Print a short table to stdout at the end of each run.

---

10. Schemas

Redis

Persistence off (`--save "" --appendonly no`). Durability difference noted in results.

```
user:{id}             HASH  username, display_name, bio, created_at
post:{id}             HASH  author_id, body, created_at, hashtags("t1,t2")
following:{id}        SET   followee ids
followers:{id}        SET   follower ids
timeline:{id}         ZSET  post_id  score=created_at_ms
tag:{tag}             ZSET  post_id  score=created_at_ms
likes:{post_id}       STRING counter (INCR)
likers:{post_id}      SET   user ids
```

- POST_CREATE: pipeline HSET post + ZADD tag:*; SMEMBERS followers (truncate to cap); pipeline ZADD timeline:{f}.
- TIMELINE_READ: ZREVRANGE timeline 0 19 → pipeline HGETALL post:* (20).
- HASHTAG_SEARCH: ZREVRANGE tag 0 19 → pipeline HGETALL.
- POST_LIKE: pipeline SADD likers + INCR likes.
- USER_FOLLOW: pipeline SADD following + SADD followers.
- USER_PROFILE_READ: HGETALL user.
- Loader: pipelines of 1000 commands.

MongoDB

Write concern `w:1`, default journaling.

```
js
```

```
users   { _id: NumberLong, username, display_name, bio, created_at }
posts   { _id: NumberLong, author_id, body, hashtags: [..], like_count, created
follows { _id: { f: follower, t: followee }, follower_id, followee_id, followed
likes   { _id: { u: user, p: post } }

posts.createIndex({ author_id: 1, created_at: -1 })
posts.createIndex({ hashtags: 1, created_at: -1 })
follows.createIndex({ follower_id: 1 })
follows.createIndex({ followee_id: 1 })
```

- TIMELINE_READ: `follows.find({follower_id}, {followee_id:1})` → `posts.find({author_id:{$in}}).sort({created_at:-1}).limit(20)`.
- POST_LIKE: insert likes (ignore duplicate key) then `$inc like_count`.
- USER_FOLLOW: insert follows (ignore duplicate key).
- Loader: `insert_many` batches of 10,000, `ordered: false`; indexes after load.

Cassandra (single node)

sql

```
CREATE KEYSPACE social_bench
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE users (user_id bigint PRIMARY KEY, username text, display_name tex
  bio text, created_at timestamp);

CREATE TABLE posts (post_id bigint PRIMARY KEY, author_id bigint, body text,
  hashtags list<text>, created_at timestamp);

CREATE TABLE posts_by_author (author_id bigint, created_at timestamp, post_id b
  body text, PRIMARY KEY (author_id, created_at, post_id))
  WITH CLUSTERING ORDER BY (created_at DESC, post_id DESC);

CREATE TABLE user_timeline (user_id bigint, created_at timestamp, post_id bigin
  author_id bigint, body text, PRIMARY KEY (user_id, created_at, post_id))
  WITH CLUSTERING ORDER BY (created_at DESC, post_id DESC);

CREATE TABLE posts_by_hashtag (hashtag text, created_at timestamp, post_id bigi
  author_id bigint, body text, PRIMARY KEY (hashtag, created_at, post_id))
  WITH CLUSTERING ORDER BY (created_at DESC, post_id DESC);

CREATE TABLE following_by_user (follower_id bigint, followee_id bigint,
  followed_at timestamp, PRIMARY KEY (follower_id, followee_id));

CREATE TABLE followers_by_user (followee_id bigint, follower_id bigint,
  followed_at timestamp, PRIMARY KEY (followee_id, follower_id));

CREATE TABLE post_likes (post_id bigint, user_id bigint, PRIMARY KEY (post_id,

CREATE TABLE post_like_counts (post_id bigint PRIMARY KEY, like_count counter);
```

- Consistency `ONE`. All statements prepared.
- POST_CREATE: insert posts, posts_by_author, posts_by_hashtag per tag; select followers_by_user (limit cap); concurrent inserts into user_timeline (`join_all`, no multi-partition batches).
- POST_LIKE: insert post_likes + counter update.
- USER_FOLLOW: insert both follow tables.
- Loader: concurrent prepared inserts, concurrency ~256.

Neo4j (Community 5)

cypher

```
CREATE CONSTRAINT user_id IF NOT EXISTS FOR (u:User) REQUIRE u.user_id IS UNIQU
CREATE CONSTRAINT post_id IF NOT EXISTS FOR (p:Post) REQUIRE p.post_id IS UNIQU
CREATE CONSTRAINT tag IF NOT EXISTS FOR (h:Hashtag) REQUIRE h.tag IS UNIQUE;
```

Nodes: `User{user_id, username, display_name, bio, created_at}`, `Post{post_id, body, created_at}`, `Hashtag{tag}`. Rels: `FOLLOWS{since}`, `POSTED`, `LIKED`, `TAGGED_WITH`. Likes exist only as `LIKED` relationships.

- TIMELINE_READ: `MATCH (:User {user_id:$uid})-[:FOLLOWS]->(a:User)-[:POSTED]->(p:Post)  RETURN p.post_id, p.body, p.created_at, a.user_id ORDER BY p.created_at DESC LIMIT 20`
- HASHTAG_SEARCH: `MATCH (:Hashtag {tag:$t})<-[:TAGGED_WITH]-(p:Post) RETURN p ORDER BY p.created_at DESC LIMIT 20`
- POST_CREATE: `MATCH (u:User {user_id:$uid}) MERGE (p:Post {post_id:$pid}) ON CREATE SET p.body=$body, p.created_at=$ts MERGE (u)-[:POSTED]->(p) WITH p UNWIND $tags AS t MERGE (h:Hashtag {tag:t}) MERGE (p)-[:TAGGED_WITH]->(h)`
- POST_LIKE / USER_FOLLOW: `MATCH` both ends, `MERGE` the relationship.
- Loader: constraints first, then `UNWIND $rows` batches of 5,000 per transaction: users → hashtags → posts → POSTED/TAGGED_WITH → FOLLOWS.

---

11. Docker Compose (Docker Desktop)

- Pin images: `redis:7.4`, `mongo:8.0`, `cassandra:5.0`, `neo4j:5.26-community`.
- One profile per database: `docker compose --profile cassandra up -d`.
- Publish ports on `127.0.0.1` only.
- Every service: `mem_limit: 4g`, `cpus: 6`, healthcheck. No `cpuset` (VM CPU numbering doesn't map to host cores).
- Cassandra env: `MAX_HEAP_SIZE=2G`, `HEAP_NEWSIZE=400M`. Health: `cqlsh -e 'describe keyspaces'`. `start_period: 120s`.
- Neo4j env: `NEO4J_AUTH=neo4j/benchpass`, `NEO4J_server_memory_heap_max__size=2G`, `NEO4J_server_memory_heap_initial__size=2G`, `NEO4J_server_memory_pagecache_size=1G`.
- Named volumes, removed with `down -v` after each database's sweep.

---

12. Sweep protocol (`scripts\run_db.ps1 -Db <db>`)

1. `sysinfo.ps1` → abort if on battery (`Win32_Battery.BatteryStatus -ne 2`); record power scheme (`powercfg /getactivescheme`).
2. `docker compose --profile <db> up -d`, poll `docker inspect` health until healthy.
3. `bench load --db <db>`.

4. Closed mode at concurrency 1, 8, 32, 128 × 3 runs.
5. Best closed-mode `ALL` throughput → open mode at 50%, 80%, 95% of it × 3 runs.
6. `docker compose --profile <db> down -v`.

`run_all.ps1`: noop calibration → redis-ping baseline (inside the redis profile) → redis → mongo → cassandra → neo4j.

Writes during runs mutate state; data is reloaded per database, not per run.

Scripts must use `$ErrorActionPreference = 'Stop'` and check `$LASTEXITCODE` after every native command.

---

13. Milestones and acceptance checks

M1 — Skeleton. Config, CLI, dataset + workload generation, `cpu.rs`. ✅ `bench run --db noop --dry-run` prints resolved config and the pinning plan with correct P/E classification; same seed → same dataset hash (unit test).

M2 — Engine + no-op. Workers, closed & open loops, rings, aggregator, output, `timeBeginPeriod`. ✅ `bench calibrate` (noop, closed, concurrency 128) reports throughput and p50/p99; dropped_records = 0; open mode at 50% of that rate reports send_lag p99 under 1 ms. ✅ Show me the exact lines where t0/t1/intended are taken and explain why nothing in the loop allocates.

M3 — Redis + transport baseline. Redis adapter, `redis_ping`, loader, compose, `run_db.ps1`. ✅ redis-ping at concurrency 1, 8, 32, 128 produces results (this is the transport floor). Full Redis sweep produces all CSVs. `docker exec` + `redis-cli` shows a new post in a follower's timeline.

M4 — MongoDB + Cassandra. ✅ Each: loader completes; one smoke run per op kind with 0 errors; full sweep runs.

M5 — Neo4j. ✅ Same as M4. `EXPLAIN` of the timeline query uses the user_id constraint index.

M6 — Plots + README. ✅ `python plots\plot.py results` produces per-op p50/p99 vs concurrency, throughput vs concurrency, and open-mode latency CDFs with all four DBs on shared axes, with the redis-ping floor drawn as a reference line on latency plots. README: one-command reproduction (`scripts\run_all.ps1`) and limitations (laptop scale, single node, Windows host + Docker VM transport overhead as measured by redis-ping, differing durability settings).

---

14. Don'ts

- No io_uring, huge pages, custom allocators, unsafe code (except the `windows` crate calls in `cpu.rs`, kept minimal and commented).
- Don't share a client/connection between workers.

- Don't `tokio::spawn` per request.
- Don't format strings or generate random numbers on the request path outside adapter calls.
- Don't silently swallow errors: count them.
- Don't "fix" slow numbers by changing the workload. Ask.